SANDIP UNIVERSITY
UGC Recognised

NAAC GRADE A

PROJECT REPORT

Threat Intelligence Aggregator

A Python tool for collecting and consolidating Indicators of Compromise (IoCs) from VirusTotal and AlienVault OTX APIs

Submitted by: _____

Programme / Department: _____

Institution: Sandip University

Academic year: 2026–2027

Abstract

Threat intelligence feeds can provide indicators that help defenders investigate suspicious activity. This project implements a command-line Python tool that collects indicators from AlienVault Open Threat Exchange (OTX) subscribed pulses and, where the account has suitable access, VirusTotal Intelligence Search. It converts provider-specific fields into a common schema, removes duplicates, retains source attribution, and exports results to JSON and CSV. API credentials are read from environment variables rather than stored in source code. The tool is intended as an educational defensive-security prototype and does not automatically block traffic or take action against any system.

1. Introduction

An Indicator of Compromise (IoC) is an observable artifact that may be associated with malicious activity. Examples include IP addresses, domains, URLs, and file hashes. Security analysts often consult more than one intelligence source, but differing data formats and duplicate entries make manual consolidation inefficient. This project demonstrates a simple, repeatable workflow for collecting and organizing IoCs.

2. Problem Statement

Analysts and students need a straightforward way to retrieve available IoCs from multiple threat-intelligence providers and store them in a consistent, analysis-friendly format. Manual copying can introduce errors, duplicate records, and weak traceability to the source.

3. Objectives

- Collect indicators from OTX subscribed pulses.
- Optionally query VirusTotal Intelligence when the account permits it.
- Normalize provider records into a shared structure.
- Deduplicate indicators while preserving provider attribution.
- Export machine-readable JSON and spreadsheet-friendly CSV.
- Keep API keys outside the code and report common API failures clearly.

4. Scope

The prototype is a command-line collector and exporter. It does not provide automated incident response, guarantee that an indicator is malicious, or replace analyst validation. Data availability depends on

provider permissions, API quotas, query support, and network connectivity. VirusTotal Intelligence Search is not guaranteed to be available with every VirusTotal API plan.

5. Tools and Technologies

• Language: Python 3.10+
• HTTP client: requests
• Data formats: JSON and CSV
• Sources: AlienVault OTX API and VirusTotal Intelligence Search API
• Configuration: environment variables
• Documentation: README and requirements sheet

6. System Design

Step 1 — Configuration: Read OTX_API_KEY and VT_API_KEY from environment variables.
Step 2 — Collection: Request subscribed OTX pulses and/or run a VirusTotal Intelligence search query.
Step 3 — Normalization: Map each provider record to fields such as indicator, type, source, pulse, first_seen, collected_at, description, and source_id.
Step 4 — Deduplication: Use lowercased indicator type and value as a key, merging source attribution for duplicates.
Step 5 — Export: Write indicators.json and indicators.csv to the selected output directory.

7. Implementation Details

The main program is threat_intelligence_aggregator.py. It uses requests to call provider endpoints and argparse to accept command-line options. API responses are checked for common failures such as authentication errors, permission restrictions, rate limiting, timeouts, and invalid JSON. A user can skip either provider using command-line flags. The default VirusTotal search query is type:ip, but the query can be changed using --vt-query or VT_SEARCH_QUERY.

8. Installation and Execution

```
python -m venv .venv
python -m pip install -r requirements.txt

# Configure keys in your shell, then run:
python threat_intelligence_aggregator.py --limit 10

# OTX only:
python threat_intelligence_aggregator.py --skip-vt

# VirusTotal only:
python threat_intelligence_aggregator.py --skip-otx --vt-query "type:ip"
```

9. Output Description

The JSON file contains a generation timestamp, total record count, and indicator records. The CSV file has one row per deduplicated indicator. The shared fields include the indicator value, provider-reported type, source, OTX pulse name where applicable, provider time where available, collection time in UTC, description, and source identifier. Missing fields are represented as empty strings.

10. Testing and Evaluation Plan

The following checks should be completed in an environment with valid API credentials: (1) verify that each provider can be skipped independently; (2) verify that missing keys produce warnings; (3) verify that invalid credentials and permission errors are reported; (4) compare JSON and CSV record counts; (5) test duplicate indicators from two sources and confirm source attribution is merged; and (6) confirm API keys do not appear in output files or source control. No live API results are claimed in this report; results depend on the user's credentials and access tier.

11. Limitations

• Provider API plans and quotas may restrict collection.
• The current version requests a limited number of results and does not implement full pagination.
• OTX pulse subscription settings affect which pulses are returned.
• VirusTotal Intelligence Search may be unavailable on a standard/free API plan.
• Collected intelligence can be stale or inaccurate and requires validation.

12. Future Enhancements

Potential improvements include pagination, incremental collection, retry/backoff, caching, unit tests using mocked responses, structured logging, scheduled runs, configurable filters, a local dashboard, and transparent confidence scoring. Any scoring should retain provenance and explain how each score was calculated.

13. Conclusion

The Threat Intelligence Aggregator provides a compact example of API integration, data normalization, deduplication, and export in Python. It helps learners understand how defensive threat-intelligence collection can be automated while highlighting practical concerns such as API permissions, error handling, credential security, and data provenance.

References

1. AlienVault Open Threat Exchange: https://otx.alienvault.com/
2. VirusTotal API documentation: https://docs.virustotal.com/reference/overview
3. Python documentation: https://docs.python.org/3/
4. Requests documentation: https://requests.readthedocs.io/

Academic prototype. Use only for authorized defensive analysis and comply with provider terms of service.